

Hamperly: Luxury Gift Hampers

Backend Engineering Evaluation: 15 Core Technical Questions

This evaluation document contains **15 technical questions** derived directly from the Java / Spring Boot backend implementation of the Hamperly Luxury Gift Hampers application. These questions are designed for code reviews, developer onboarding, and technical assessment, focusing on security, payment processing, transactional integrity, and database analytics.

Question 1: Spring Security Config & Public vs. Private API Endpoints

In SecurityConfig.java, what is the configuration used to specify which endpoints are publicly accessible (permit all) and which endpoints require administrative role checks (hasRole)? Additionally, what annotation is placed at the configuration class level to enable method-level security checking across service layers, and how is the JwtAuthenticationFilter integrated relative to Spring Security's standard filters?

Key Files:	SecurityConfig.java
Concepts Covered:	HttpSecurity, SecurityFilterChain, permitAll, hasRole('ADMIN'), @EnableMethodSecurity, addFilterBefore

Question 2: JWT Signing Key Creation and Cryptographic Algorithm

In JwtTokenProvider.java, inspect the private helper method getSigningKey(). How is the signing key created from the application secret, and what encryption algorithm does the underlying Keys.hmacShaKeyFor library use to sign the JWT? Why is it crucial that the jwtSecret configured in application.properties meets a minimum byte size requirement for this algorithm?

Key Files:	JwtTokenProvider.java
Concepts Covered:	HMAC-SHA256, Keys.hmacShaKeyFor, SecretKey, Base64 & UTF-8 Encoding

Question 3: Fallback Redaction Logic in Email Services

In EmailServiceImpl.java, how does the system handle the password reset flow when the JavaMailSender bean is not configured (e.g., set to null)? Describe the specific string-based mechanism used to redact the security token before outputting the warning logs. What is the security purpose of this fallback redaction logic?

Key Files:	EmailServiceImpl.java
Concepts Covered:	Optional Bean Injection, Null Checks, Token Obfuscation, Logging Safety

Question 4: Cryptographically Secure Password Reset Tokens

In PasswordResetServiceImpl.java, what class is utilized in the generateSecureToken() method to ensure cryptographic randomness? Furthermore, describe how the generated raw token is hashed using SHA-256 in the hashToken() method before storing it in the database. Why is storing only the hash in the database considered a security best practice compared to storing the raw token?

Key Files:	PasswordResetServiceImpl.java
Concepts Covered:	SecureRandom, Base64 URL Encoder, MessageDigest SHA-256, One-Way Hashing

Question 5: Active Token Invalidation on Request Creation & Execution

In PasswordResetServiceImpl.java, how does the application handle previously generated active reset tokens belonging to a user when a new forgot password request is submitted? Explain how this invalidation is triggered both at request generation (forgot-password) and after a successful password update (reset-password). Which JPA repository method is invoked to accomplish this?

Key Files:	PasswordResetServiceImpl.java, PasswordResetTokenRepository.java
Concepts Covered:	Token Replay Protection, Token Lifecycle, JPA Custom Query Invalidation

Question 6: Password Strength Policy and Weak Password Blacklisting

In PasswordResetServiceImpl.java, analyze the validatePasswordStrength() method. What specific regular expressions are checked to ensure password complexity? Additionally, explain how the system detects and rejects commonly used weak passwords (e.g., 'password123', 'admin123'). What exception classes are thrown when these checks fail?

Key Files:	PasswordResetServiceImpl.java
Concepts Covered:	Regex Patterns, Password Policies, Set Membership Blacklists, WeakPasswordException

Question 7: Database Environment Integration and Schema Update Policy

In application.properties, describe the syntax used to import environment variables from the external .env file. What is the value configured for spring.jpa.hibernate.ddl-auto? Explain the implications of this configuration setting on both the development database and a production database environment.

Key Files:	application.properties, .env.example
Concepts Covered:	spring.config.import, Hibernate DDL Auto Policies, Database Migration Best Practices

Question 8: Invoice Generation Calculations & GST Component Extraction

In InvoiceGenerator.java, what open-source PDF generator library is used (com.lowagie.text)? Given that item prices on the platform are inclusive of an 18% GST rate, inspect the formula used to extract and display the individual GST component for each order item. How is rounding handled during this decimal division in Java?

Key Files:	InvoiceGenerator.java
Concepts Covered:	OpenPDF Library, GST Calculation Math, BigDecimal Scaling & Rounding (ROUND_HALF_UP)

Question 9: Transactional Status Constraints for Invoices

In `OrderServiceImpl.java`, examine the `getOrderInvoicePath()` method. Under what exact order status condition is the system permitted to generate and save an invoice? What error is thrown if a client attempts to retrieve or download an invoice PDF while the order is still in a 'CONFIRMED' or 'SHIPPED' status?

Key Files: *OrderServiceImpl.java, InvoiceRepository.java*

Concepts Covered: *OrderStatus Enum Guard Clauses, IllegalArgumentException, Lazy Invoice Generation*

Question 10: Razorpay Integration and Payment Signature Verification

In `OrderServiceImpl.java`, during `createOrder()`, why is the checkout grand total multiplied by 100 before passing it to Razorpay's client order constructor? Additionally, explain how the system verifies that the subsequent payment notification is legitimate and has not been spoofed, identifying the parameters verified in `verifyOrderPayment()` using the Razorpay SDK's utility classes.

Key Files: *OrderServiceImpl.java*

Concepts Covered: *Razorpay Java SDK, Currency subunit conversion (paise), Utils.verifyPaymentSignature, Signature Spoofing Prevention*

Question 11: Stock Availability Checks & Concurrency Controls

In `OrderServiceImpl.java`, stock validation happens during two separate methods: `createOrder()` and `verifyOrderPayment()`. Why is this double check necessary? Discuss potential issues in a concurrent environment where two users try to purchase the last available stock of a hamper simultaneously. How does the Spring `@Transactional` boundary affect the consistency of stock levels?

Key Files: *OrderServiceImpl.java, Product.java*

Concepts Covered: *Double-check validation, Concurrency race conditions, Transaction boundaries, JPA Locking (Pessimistic vs Optimistic)*

Question 12: Native SQL Queries vs. JPQL for Analytics Reports

In `AdminOrderRepository.java`, look at the annotations for `findHourlyRevenue`, `findDailyRevenueForMonth`, and `findMonthlyRevenueForYear`. Why are these queries declared as native queries (`nativeQuery = true`) instead of using standard JPQL/HQL? Identify the database-specific functions leveraged in these SQL statements that restrict portability.

Key Files: *AdminOrderRepository.java, AdminAnalyticsServiceImpl.java*

Concepts Covered: *Native SQL vs JPQL, Database-specific Date/Time Functions (HOUR, DATE, YEAR, MONTH), Query aggregation*

Question 13: CORS Security Settings and Development Wildcards

In SecurityConfig.java, review the corsConfigurationSource() bean. What configuration settings have been applied to allowedOriginPatterns, allowedMethods, and allowedHeaders to facilitate development? Explain why configuring wildcards (*) for these properties in combination with allowCredentials(true) is a security risk in production environments (OWASP TOP 10 CORS vulnerabilities).

Key Files:	SecurityConfig.java
Concepts Covered:	Cross-Origin Resource Sharing (CORS), Origin Wildcards, Credentials sharing security, Pre-flight requests

Question 14: Lifecycle Timing of Shopping Cart Clearance

In the checkout workflow in OrderServiceImpl.java, at what exact phase in the transaction is the customer's cart cleared in the database (i.e. deleting items from CartItemRepository)? Why is it crucial that the cart is NOT cleared during the initial createOrder() call before payment is processed?

Key Files:	OrderServiceImpl.java, CartItemRepository.java
Concepts Covered:	Cart lifecycle, Order state transitions, Transaction persistence order, UX protection against dropped payments

Question 15: Order Tracking Progress Compilation and Timestamps

In OrderServiceImpl.java, describe how the getOrderTracking() method dynamically maps the tracking timeline (Pending, Confirmed, Packed, Shipped, Out for Delivery, Delivered). How does it evaluate if a step is completed, and how are the respective timestamps mapped from the Order entity's audit fields?

Key Files:	OrderServiceImpl.java, Order.java
Concepts Covered:	Tracking status progression, Conditional step creation, Audit timestamps Mapping, DTO Transformation